ASSIGNMENT-1

2303A52332

TASK 0: Install & Configure GitHub Copilot (MANDATORY)

Step-by-Step

Open VS Code

Go to Extensions

Search GitHub Copilot

Click Install

Sign in using your GitHub account

Restart VS Code if asked

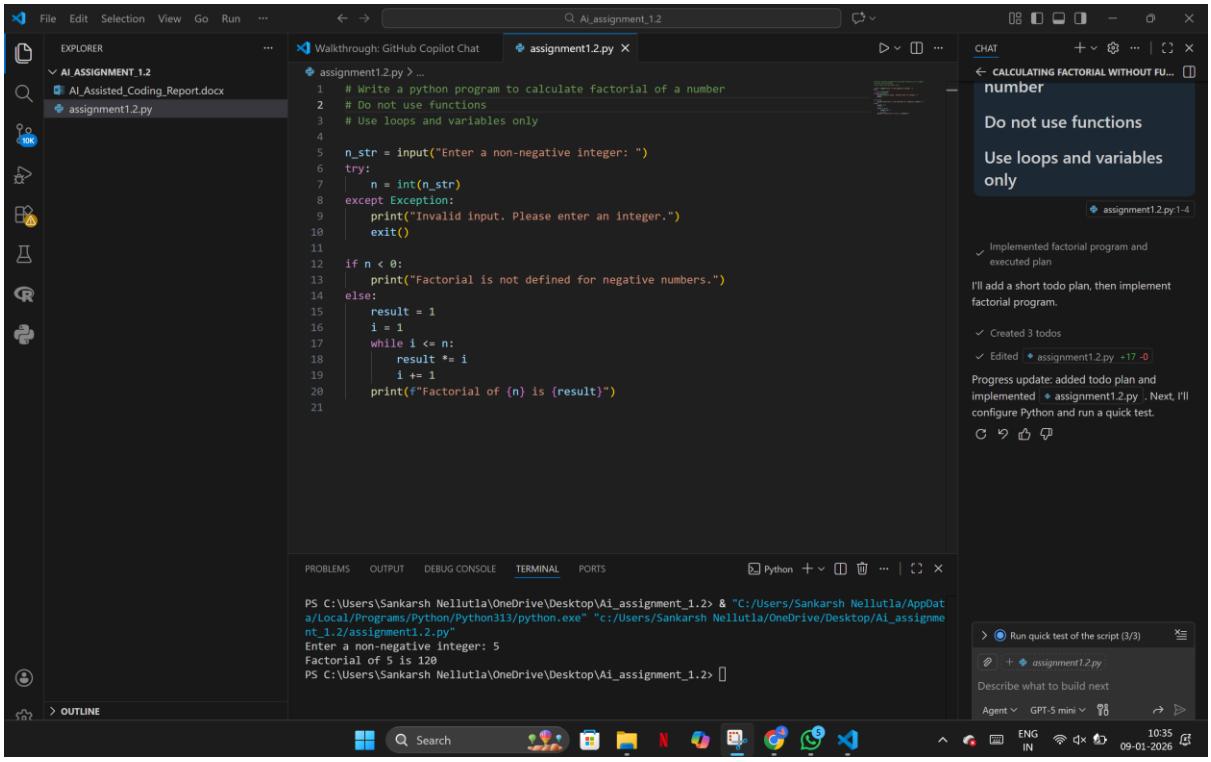
Screenshots to Take

Copilot extension page

GitHub sign-in success

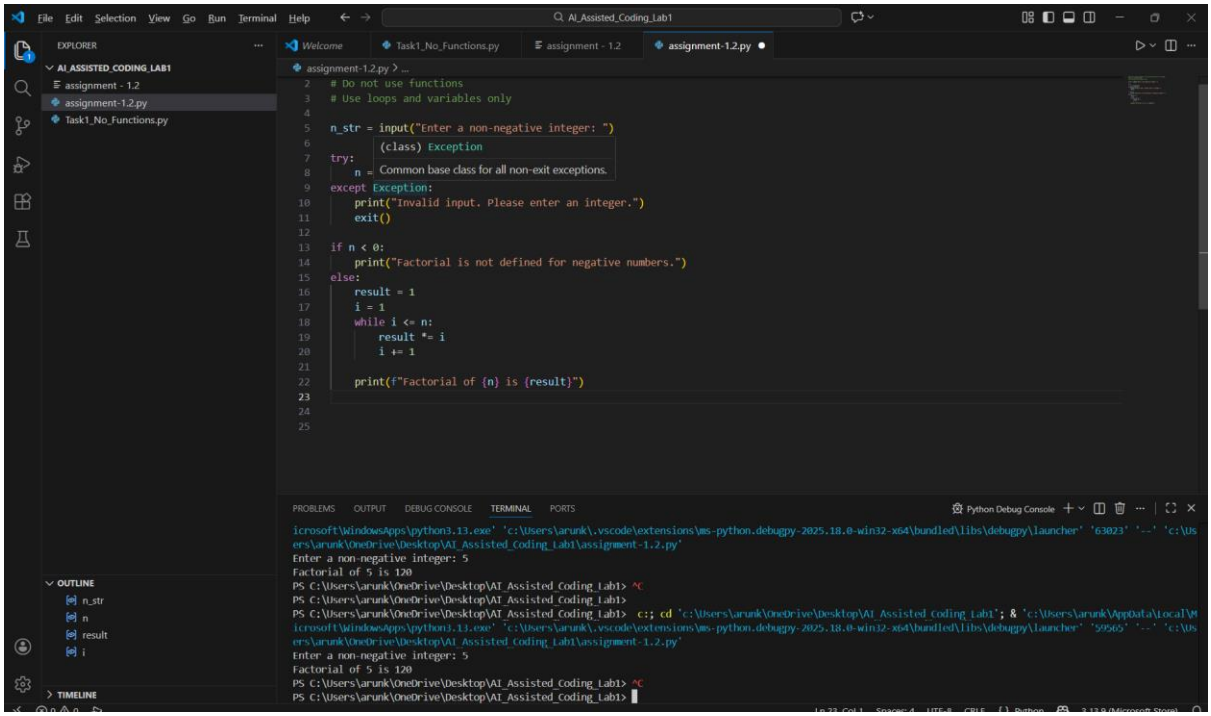
Copilot enabled in VS Code

TASK 1: AI-Generated Logic (NO FUNCTIONS)



Task-2: Optimized version

TASK 3: Modular Design (WITH FUNCTIONS)



Task 4: Comparative Analysis – Procedural vs Modular AI Code

Introduction

In this task, we compare two AI-generated Python programs created using GitHub Copilot:

1. A **procedural version** (without functions)
2. A **modular version** (with user-defined functions)

Both programs compute the factorial of a number, but they differ in structure and design. This comparison helps justify design choices during code reviews and understand best practices in AI-assisted coding.

Comparison Table

Criteria	Procedural Code (Without Functions)	Modular Code (With Functions)
Logic Clarity	Logic is written in a single block, making it slightly harder to understand for larger programs	Logic is clearly separated into a function, making it easier to read and follow
Reusability	Cannot be reused easily; code must be rewritten if needed elsewhere	Highly reusable; the function can be called from multiple programs
Ease of Debugging	Debugging is harder since all logic is mixed together	Easier debugging because errors can be isolated inside the function
Suitability for Large Projects	Not suitable for large projects due to poor structure	Very suitable for large projects due to modular design
AI Dependency Risk	Higher risk because AI-generated code may be copied blindly without structure	Lower risk because modular code encourages review and refinement

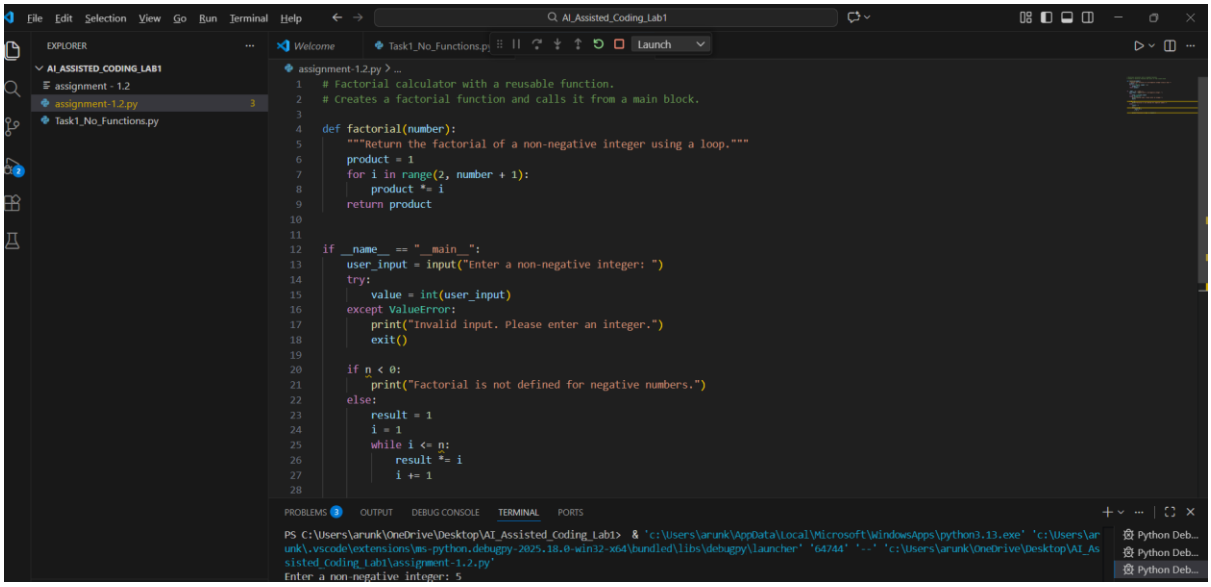
Conclusion

The procedural approach is useful for quick, small programs or beginner-level tasks where speed is more important than structure. However, the modular approach is clearly superior for real-world applications. Function-based programs improve readability, reusability, maintainability, and debugging efficiency.

From an AI-assisted coding perspective, modular design also reduces dependency risk because developers are encouraged to understand and validate each function rather than blindly accepting AI-generated output. Therefore, for scalable and professional software development, the modular approach is strongly recommended.

TASK 5: Iterative vs Recursive AI Thinking

Iterative:


File Edit Selection View Go Run Terminal Help
AI_Assisted_Coding_Lab1
EXPLORER
AI_ASSISTED_CODING_LAB1
assignment - 1.2
assignment-1.2.py
Task1_No_Functions.py
Welcome
Task1_No_Functions.py
Launch
assignment-1.2.py > ...
# Factorial calculator with a reusable function.
# Creates a factorial function and calls it from a main block.
def factorial(number):
"""Return the factorial of a non-negative integer using a loop."""
product = 1
for i in range(2, number + 1):
product *= i
return product
if __name__ == "__main__":
user_input = input("Enter a non-negative integer: ")
try:
value = int(user_input)
except ValueError:
print("Invalid input. Please enter an integer.")
exit()
if n < 0:
print("Factorial is not defined for negative numbers.")
else:
result = 1
i = 1
while i <= n:
result *= i
i += 1
PROBLEMS 3
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\arunk\OneDrive\Desktop\AI_Assisted_Coding_Lab1> & 'c:\Users\arunk\AppData\Local\Microsoft\WindowsApps\python3.13.exe' 'c:\Users\arunk\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '64744' '--' 'c:\Users\arunk\OneDrive\Desktop\AI_Assisted_Coding_Lab1\assignment-1.2.py'
Enter a non-negative integer: 5
Python Deb...
Python Deb...
Python Deb...

Recursive: